

1 개요

이번 강의는 데이터 엔지니어링의 핵심인 데이터 파이프라인에 초점을 맞춥니다. 데이터 파이프라인은 데이터를 한 지점에서 다른 지점으로 이동시키거나, 여러 소스에서 데이터를 수집하고 변환하여 다양한 목적지로 저장하는 과정을 자동화하는 시스템입니다. 특히 ETL(Extract, Transform, Load) 및 ELT(Extract, Load, Transform)와 같은 특정 유형의 파이프라인을 포함하여, 배치(Batch) 및 스트리밍(Streaming) 처리 방식의 차이점과 필요성을 깊이 있게 탐구합니다.

강의는 이전 수업에서 다룬 개념들을 복습하는 것으로 시작하여, 스트리밍 데이터 처리의 필요성과 기본 개념을 소개합니다. 이후 배치와 스트리밍 처리의 장단점을 비교하고, Lambda 및 Kappa 아키텍처와 같은 데이터 파이프라인 설계 패턴을 상세히 설명합니다. 또한, 데이터 파이프라인에서 발생할 수 있는 실패, 트레이드오프, 그리고 모범 사례에 대해서도 논의합니다.

강의의 후반부에는 Lab 02 실습을 통해 실제 네트워크 로그 데이터를 파싱하고 분석하는 과정을 직접 경험합니다. 이를 통해 이론적 지식을 실제 Databricks 환경에서 어떻게 적용하는지 배울 수 있습니다. 마지막으로, 과제 2에 대한 안내와 함께 질의응답 시간을 가집니다.

2 용어 정리

데이터 엔지니어링 분야에서 자주 사용되는 핵심 용어들을 정리했습니다. 각 용어의 쉬운 설명과 함께 원어 및 비고를 통해 이해를 돕습니다.

용어	쉬운 설명	원어	비고
3차 정규형	데이터 중복을 줄이고 무결성을 높이는 관계형 DB 설계 원칙	Third Normal Form (3NF)	관계형 데이터베이스에 주로 적용
비정규화	쿼리 성능 향상을 위해 데이터를 중복 저장하거나 통합하는 과정	Denormalization	NoSQL/비관계형 데이터베이스에 주로 적용
ETL	데이터 추출, 변환, 적재의 세 단계로 이루어진 데이터 처리 과정	Extract, Transform, Load	전통적인 배치 처리 방식
ELT	데이터 추출, 적재 후 변환하는 과정	Extract, Load, Transform	클라우드 환경에서 유연한 처리 가능
스타 스키마	중앙의 팩트 테이블과 주변의 차원 테이블로 구성된 데이터 웨어하우스 모델	Star Schema	단순하고 쿼리 성능이 좋음
스노우플레이크 스키마	스타 스키마에서 차원 테이블이 추가로 정규화된 형태	Snowflake Schema	데이터 중복이 적고 복잡도가 높음
카-값 저장소	고유한 키와 해당 키에 연결된 값으로 데이터를 저장하는 방식	Key-Value Store	S3, Redis 등. 단순하고 확장성이 뛰어남
문서 저장소	JSON, XML 등 문서 형태로 데이터를 저장하는 방식	Document Store	MongoDB, CouchDB 등. 유연한 스키마
컬럼형 저장소	데이터를 행이 아닌 컬럼 단위로 저장하여 분석 쿼리 성능을 높이는 방식	Columnar Data Store	Cassandra, DynamoDB 등. OLAP에 적합
Parquet	컬럼형 바이너리 데이터 저장 형식	Parquet	Apache Parquet. 효율적인 압축 및 쿼리 성능
Delta Lake	데이터 레이크에 ACID 트랜잭션, 스키마 관리 등을 제공하는 오픈 소스 저장 계층	Delta Lake	Databricks에서 개발. 배치/스트리밍 통합
메타데이터	데이터에 대한 데이터. 데이터의 맥락과 구조를 설명	Metadata	스키마, 트랜잭션 로그 등
뷰	쿼리 정의를 저장하고, 참조 시마다 쿼리를 실행하여 결과를 생성하는 가상 테이블	View (Virtual Table)	실제 데이터를 저장하지 않음
구체화된 뷰	쿼리 결과를 미리 계산하여 저장해두는 뷰	Materialized View	쿼리 성능 향상. 데이터가 오래될 수 있음
데이터 파이프라인	데이터의 흐름을 자동화하고 관리하는 일련의 과정	Data Pipeline	데이터 통합 및 엔지니어링의 핵심
배치 처리	일정량의 데이터를 모아 한 번에 처리하는 방식	Batch Processing	보고서, 청구서 처리 등
스트리밍 처리	데이터가 생성되는 즉시 실시간으로 처리하는 방식	Streaming Processing	사기 탐지, 실시간 대시보드 등
마이크로 배치	스트리밍 데이터를 작은 배치 단위로 나누어 처리하는 방식	Microbatch	Spark Structured Streaming의 기본 모드
체크포인트	스트리밍 처리 중 마지막으로 성공적으로 처리된 지점을 기록	Checkpoint	장애 발생 시 복구 지점
워터마크	스트리밍 데이터에서 지연 도착 데이터를 처리하는 기준 시간	Watermark	너무 늦게 도착한 데이터는 무시
Lambda 아키텍처	배치 레이어와 스트리밍 레이어를 분리하여 운영하는 데이터 처리 아키텍처	Lambda Architecture	초기 스트리밍 시스템에서 많이 사용
Kappa 아키텍처	모든 데이터를 스트리밍으로 처리하는 단일 레이어 아키텍처	Kappa Architecture	Lambda의 복잡성을 줄인 현대적 방식
Kafka	분산 스트리밍 플랫폼	Kafka	고성능 메시지 큐 및 스트리밍 처리
Flink	진정한 실시간 스트리밍 처리 프레임워크	Flink	매우 낮은 지연 시간, 복잡 이벤트 처리 (CEP)
Spark Structured Streaming	Apache Spark 기반의 스트리밍 처리 API	Spark Structured Streaming	배치/스트리밍 통합 API, Lakehouse의 기반
Unity Catalog	Databricks의 통합 데이터 거버넌스 솔루션	Unity Catalog	메타데이터 관리, 접근 제어, 데이터 계보
Photon	Databricks의 고성능 Spark 엔진	Photon	C++로 제작되어 빠른 쿼리 성능 제공
UDF	사용자가 직접 정의하여 재사용할 수 있는 함수	User-Defined Function	데이터 변환 로직을 캡슐화

3 핵심 개념 및 원리

3.1 데이터베이스 정규화 및 비정규화

정규화(Normalization)는 관계형 데이터베이스의 중복을 줄이고 데이터 무결성을 높이는 과정입니다. 반대로 **비정규화(Denormalization)**는 쿼리 성능 향상을 위해 의도적으로 데이터를 중복시키거나 통합하는 과정입니다.

3.1.1 3차 정규형 (Third Normal Form, 3NF)

- **한 줄 핵심 요약:** 데이터 중복을 최소화하고 데이터의 일관성을 유지하기 위한 관계형 데이터베이스 설계 규칙입니다.
- **직관적 비유:** 도서관에서 책 정보를 관리할 때, 모든 책의 저자 정보(이름, 생년월일, 국적)를 각 책 레코드마다 반복해서 적는 대신, 저자 정보를 별도의 목록으로 만들고 책 레코드에서는 저자 ID만 참조하는 것과 같습니다. 이렇게 하면 저자 정보가 바뀌어도 한 곳만 수정하면 됩니다.
- **기술적 설명:**
 - **목표:** 데이터 중복을 줄이고 데이터 무결성(Data Integrity)을 보장합니다.
 - **적용:** 주로 관계형 데이터베이스(Relational Databases)에서 사용됩니다.
 - **원리:** 테이블을 여러 개의 작은 테이블로 분리하고, 외래 키(Foreign Key)를 사용하여 테이블 간의 관계를 정의합니다.

3.1.2 비정규화 (Denormalization)

- **한 줄 핵심 요약:** 쿼리 성능을 극대화하기 위해 데이터 중복을 허용하고 여러 테이블의 데이터를 하나의 테이블로 결합하는 전략입니다.
- **직관적 비유:** 도서관에서 특정 주제의 책 목록을 자주 찾아야 할 때, 매번 책 정보와 저자 정보를 따로 찾아 결합하는 것이 번거롭다면, 아예 자주 찾는 책 목록에는 저자 이름까지 함께 기록해두는 것과 같습니다. 이렇게 하면 검색은 빨라지지만, 저자 정보가 바뀌면 여러 곳을 수정해야 하는 단점이 있습니다.
- **기술적 설명:**
 - **목표:** 조인(Join) 작업의 필요성을 줄여 쿼리 성능을 향상시킵니다.
 - **적용:** NoSQL 또는 비관계형 데이터베이스(Non-Relational Databases)에서 주로 사용됩니다. 조인 기능이 없거나 제한적인 경우에 특히 유용합니다.
 - **원리:** 여러 테이블의 측정값(measures)을 하나의 테이블로 결합하여 정보의 중복을 허용합니다.

Table 1: 정규화 vs 비정규화 비교

특징	정규화	비정규화
목표	데이터 중복 감소, 무결성 보장	쿼리 성능 향상
데이터 중복	최소화	허용 (의도적)
테이블 구조	여러 개의 작은 테이블	통합된 큰 테이블
주요 사용처	관계형 데이터베이스 (OLTP)	NoSQL, 데이터 웨어하우스 (OLAP)
조인 필요성	높음	낮음
데이터 일관성	높음	낮을 수 있음 (중복 데이터 관리 필요)

3.2 ETL과 ELT

ETL(Extract, Transform, Load)은 데이터를 추출, 변환, 적재하는 전통적인 방식이며, **ELT(Extract, Load, Transform)**는 데이터를 추출, 적재한 후 변환하는 방식입니다.

3.2.1 ETL (Extract, Transform, Load)

- **한 줄 핵심 요약:** 소스 시스템에서 데이터를 추출(Extract)하여 필요한 형태로 변환(Transform)한 후, 대상 시스템에 적재(Load)하는 과정입니다.
- **직관적 비유:** 해외에서 수입한 원자재(데이터)를 국내 공장(변환 시스템)에서 가공하여 완제품(변환된 데이터)을 만든 후, 창고(대상 시스템)에 보관하는 것과 같습니다.
- **기술적 설명:**
 - **Extract:** 다양한 소스(데이터베이스, 파일 등)에서 데이터를 가져옵니다.
 - **Transform:** 데이터 품질 향상, 형식 통일, 집계, 정규화 등 분석에 적합한 형태로 데이터를 변경합니다. 이 단계는 대상 시스템에 적재하기 전에 이루어집니다.
 - **Load:** 변환된 데이터를 데이터 웨어하우스나 데이터 마트와 같은 대상 시스템에 저장합니다.
 - **특징:** 주로 전통적인 배치 지향(Batch-oriented) 처리 방식에 사용되며, 들어오는 데이터의 스키마(Schema)가 비교적 잘 알려져 있을 때 효과적입니다.

3.2.2 ELT (Extract, Load, Transform)

- **한 줄 핵심 요약:** 소스 시스템에서 데이터를 추출(Extract)하여 대상 시스템에 먼저 적재(Load)한 후, 대상 시스템 내에서 변환(Transform)하는 과정입니다.
- **직관적 비유:** 해외에서 수입한 원자재(데이터)를 일단 국내 창고(대상 시스템)에 그대로 보관한 후, 필요할 때마다 창고 내에서 가공하여 사용하는 것과 같습니다.
- **기술적 설명:**
 - **Extract:** 다양한 소스에서 데이터를 가져옵니다.
 - **Load:** 원본 데이터를 거의 그대로 데이터 레이크(Data Lake)와 같은 대상 시스템에 저장합니다.
 - **Transform:** 대상 시스템(주로 클라우드 기반의 확장 가능한 스토리지 및 컴퓨팅 자원) 내에서 데이터를 변환합니다. 이 방식은 클라우드 환경의 탄력적인 자원을 활용하여 대규모 데이터 처리 및 유연한 스키마 변경에 유리합니다.
 - **특징:** 들어오는 데이터의 스키마가 덜 알려져 있거나, 나중에 다양한 방식으로 데이터를 분석하고 싶을 때 유용합니다. 원본 데이터를 보존하면서 필요에 따라 여러 변환을 적용할 수 있습니다.

3.2.3 데이터 웨어하우스 및 데이터 마트 스키마

데이터 웨어하우스와 데이터 마트는 분석 및 보고를 위해 데이터를 저장하는 시스템입니다. 이들은 주로 두 가지 스키마를 사용합니다.

- **스타 스키마 (Star Schema):**
 - 중앙에 하나의 팩트 테이블(Fact Table)이 있고, 그 주위를 여러 차원 테이블(Dimension Table)이 둘러싸는 형태입니다.
 - 팩트 테이블은 측정값(예: 판매량, 금액)을 포함하고, 차원 테이블은 측정값의 맥락(예: 시간, 제품, 고객)을 제공합니다.

- 단순하고 이해하기 쉬우며, 쿼리 성능이 좋습니다.
- **스노우플레이크 스키마 (Snowflake Schema):**
 - 스타 스키마와 유사하지만, 차원 테이블이 추가로 정규화되어 여러 계층의 차원 테이블을 가질 수 있습니다.
 - 데이터 중복이 적지만, 조인(Join)이 더 많이 필요하여 쿼리 복잡도가 높아지고 성능에 영향을 줄 수 있습니다.

3.3 데이터 저장소 유형

데이터는 다양한 방식으로 저장될 수 있으며, 각 방식은 특정 사용 사례에 최적화되어 있습니다.

3.3.1 키-값 저장소 (Key-Value Store)

- **한 줄 핵심 요약:** 고유한 키(Key)와 해당 키에 연결된 값(Value)으로 데이터를 저장하는 가장 단순한 형태의 데이터 저장소입니다.
- **직관적 비유:** 사물함에 물건을 보관할 때, 사물함 번호(키)와 그 안에 들어있는 물건(값)을 매칭하여 저장하는 것과 같습니다. 번호를 알면 즉시 물건을 찾을 수 있습니다.
- **기술적 설명:**
 - **예시:** JSON 파일, Amazon S3 (객체 스토리지).
 - **특징:** 매우 단순한 저장 방식이지만, 높은 확장성(Scalability)과 빠른 접근 속도(Fast Access)를 제공합니다. 값은 어떤 형태의 객체(Object)도 될 수 있습니다.

3.3.2 문서 저장소 (Document Store)

- **한 줄 핵심 요약:** 데이터를 JSON, XML, BSON과 같은 문서(Document) 형태로 저장하는 데이터 저장소입니다.
- **직관적 비유:** 서류 캐비닛에 각 서류철(문서)을 통째로 보관하는 것과 같습니다. 각 서류철은 다양한 정보를 포함할 수 있으며, 필요에 따라 서류철의 내용을 유연하게 변경할 수 있습니다.
- **기술적 설명:**
 - **예시:** MongoDB, CouchDB.
 - **특징:** 키-값 저장소보다 더 복잡한 구조의 데이터를 저장할 수 있으며, 스키마가 유연하여 데이터 모델 변경에 용이합니다.

3.3.3 컬럼형 데이터 저장소 (Columnar Data Store)

- **한 줄 핵심 요약:** 데이터를 행(Row) 단위가 아닌 컬럼(Column) 단위로 저장하여 분석 쿼리 성능을 최적화하는 데이터 저장소입니다.
- **직관적 비유:** 일반적인 스프레드시트가 행별로 데이터를 저장한다면, 컬럼형 저장소는 각 열(컬럼)의 데이터를 한데 모아 저장하는 방식입니다. 특정 열의 데이터만 필요할 때 전체 행을 읽을 필요 없이 해당 열만 빠르게 읽을 수 있습니다.
- **기술적 설명:**
 - **예시:** Cassandra, DynamoDB.
 - **특징:** 관계형 데이터베이스가 행 단위로 데이터를 저장하는 것과 달리, 컬럼 단위 저장은 특정

컬럼에 대한 집계(Aggregation) 쿼리나 분석 작업에서 뛰어난 성능을 발휘합니다.

- **왜 필요한가?:** 데이터 분석 시에는 특정 몇몇 컬럼의 데이터만 필요한 경우가 많습니다. 예를 들어, '총 판매액'을 계산할 때는 '판매 금액' 컬럼만 필요하고, '고객 이름'이나 '주소' 컬럼은 필요하지 않습니다. 컬럼형 저장소는 이처럼 필요한 컬럼만 효율적으로 읽어올 수 있어 I/O(입출력) 비용을 크게 줄이고 쿼리 속도를 향상시킵니다.

3.4 데이터 형식: 바이너리 vs 텍스트, Parquet

3.4.1 바이너리 형식 (Binary Formats) 선호 이유

- **한 줄 핵심 요약:** 바이너리 형식은 텍스트 형식보다 데이터 압축률이 높고 읽기/쓰기 속도가 빠르며, 전반적인 성능이 우수합니다.
- **직관적 비유:** 책을 보관할 때, 모든 내용을 손으로 직접 쓰는 것(텍스트)보다 인쇄된 책(바이너리)이 훨씬 작고 가벼우며, 읽기도 빠르고 보관도 효율적인 것과 같습니다.
- **기술적 설명:**
 - **압축률:** 바이너리 형식은 데이터를 더 효율적으로 압축할 수 있어 저장 공간을 절약합니다.
 - **성능:** 데이터 파싱(Parsing) 과정이 텍스트보다 단순하여 읽기 및 쓰기 작업이 더 빠릅니다.
 - **데이터 타입:** 숫자, 날짜 등 다양한 데이터 타입을 텍스트 변환 없이 원시 형태로 저장하여 정확성과 효율성을 높입니다.

3.4.2 Parquet 형식

- **한 줄 핵심 요약:** Parquet은 컬럼형(Columnar) 바이너리 데이터 저장 형식으로, 높은 압축률과 빠른 쿼리 성능을 제공하여 빅데이터 분석에 매우 적합합니다.
- **직관적 비유:** 도서관에서 책을 보관할 때, 모든 책을 한 줄로 쪽 늘어놓는 대신, 같은 종류의 책(예: 소설, 과학, 역사)끼리 모아서 별도의 서가에 보관하는 것과 같습니다. 특정 종류의 책만 찾을 때 훨씬 빠르게 접근할 수 있습니다.
- **기술적 설명:**
 - **형식:** 컬럼형 바이너리 형식입니다.
 - **장점:**
 - * **효율적인 압축:** 컬럼별로 데이터 타입이 유사하여 높은 압축률을 달성합니다.
 - * **빠른 쿼리 성능:** 필요한 컬럼만 읽어올 수 있어 I/O 비용을 줄이고 쿼리 속도를 향상시킵니다.
 - * **스키마 진화(Schema Evolution):** 스키마 변경에 유연하게 대응할 수 있습니다.

3.5 Delta Lake 형식

Delta Lake는 데이터 레이크에 ACID 트랜잭션, 스키마 관리, 시간 여행(Time Travel) 등의 기능을 추가하여 데이터의 신뢰성과 성능을 높이는 오픈 소스 저장 계층입니다.

- **한 줄 핵심 요약:** Databricks에서 제공하는 오픈 소스 테이블 형식으로, 데이터 레이크에 데이터 웨어하우스의 안정성과 신뢰성을 부여합니다.
- **직관적 비유:** 일반적인 데이터 레이크가 정돈되지 않은 창고라면, Delta Lake는 창고에 재고 관리 시스템, 보안 시스템, 버전 관리 시스템을 도입하여 물건을 더 안전하고 효율적으로 관리할 수 있게

해주는 것과 같습니다.

- 기술적 설명:

- **제공처:** Databricks에서 개발한 오픈 소스 테이블 형식입니다.

- **주요 속성 (4가지):**

1. **ACID 트랜잭션 (Atomicity, Consistency, Isolation, Durability):** 데이터의 안정성과 일관성을 보장합니다. 여러 사용자가 동시에 데이터를 읽고 써도 데이터 손상이나 불일치가 발생하지 않습니다.
2. **스키마 진화 (Schema Evolution):** 데이터 스키마 변경에 유연하게 대응할 수 있습니다. 새로운 컬럼 추가, 기존 컬럼 타입 변경 등을 쉽게 처리합니다.
3. **데이터 이력 (History) / 시간 여행 (Time Travel):** 데이터의 모든 변경 이력을 기록하여 특정 시점의 데이터 상태로 되돌아가거나(롤백), 과거 데이터를 조회할 수 있습니다.
4. **배치 및 스트리밍 모드 통합 (Unifying Batch and Streaming Modes):** 배치 처리와 스트리밍 처리를 동일한 테이블에서 일관된 방식으로 수행할 수 있습니다.

- **추가 기능:**

- * **세분화된 삽입 및 삭제 (Fine-grained Inserts and Deletes):** 데이터베이스 내의 데이터를 매우 세분화된 수준에서 쉽게 업데이트하거나 삭제할 수 있습니다.
- * **데이터 버전 관리 (Versioning):** 데이터의 여러 버전을 관리하여 이전 버전으로 돌아가거나 앞으로 이동할 수 있습니다.

3.6 메타데이터 (Metadata)

메타데이터는 '데이터에 대한 데이터'를 의미하며, 데이터의 맥락과 구조를 이해하는 데 필요한 정보를 제공합니다.

- **한 줄 핵심 요약:** 데이터 자체의 내용이 아니라, 데이터가 무엇인지, 어떻게 구성되어 있는지, 언제 생성되었는지 등을 설명하는 정보입니다.
- **직관적 비유:** 책의 내용(데이터)이 아니라, 책의 제목, 저자, 출판일, 페이지 수, 목차(메타데이터)와 같습니다. 이 정보들을 통해 책의 내용을 파악하고 관리할 수 있습니다.
- **기술적 설명:**
 - **역할:** 데이터의 컨텍스트(Context)를 제공하여 데이터를 이해하고 활용하는 데 도움을 줍니다.
 - **예시:**
 - * **스키마 (Schemas):** 테이블의 컬럼 이름, 데이터 타입, 제약 조건 등 데이터의 구조를 정의합니다.
 - * **트랜잭션 로그 (Transaction Logs):** 데이터베이스에서 발생한 모든 변경 사항을 기록합니다.
 - * **Delta Lake의 로그:** Delta Lake 테이블의 모든 변경 이력과 버전을 관리하는 데 사용됩니다.

3.7 테이블 유형

데이터베이스 시스템에서는 다양한 목적과 수명 주기를 가진 테이블 유형을 제공합니다.

3.7.1 영구 테이블 (Permanent Tables)

- **한 줄 핵심 요약:** 데이터의 영속성(Persistence)을 지원하며, 명시적으로 삭제하기 전까지 데이터베이스에 계속 저장됩니다.

- **직관적 비유:** 은행 계좌 정보처럼 한 번 생성되면 계속 유지되고, 필요할 때마다 조회하거나 업데이트 할 수 있는 데이터입니다.

3.7.2 임시 테이블 (Temporary Tables)

- **한 줄 핵심 요약:** 특정 세션(Session) 또는 트랜잭션(Transaction) 동안만 존재하며, 세션이 종료되면 자동으로 삭제되는 테이블입니다.
- **직관적 비유:** 웹사이트에서 장바구니에 담은 상품 목록처럼, 웹 브라우저를 닫으면 사라지는 임시 정보와 같습니다.
- **유형:**
 - **로컬 임시 테이블 (Local Temporary Tables):** 현재 세션 동안만 존재합니다.
 - **글로벌 임시 테이블 (Global Temporary Tables):** 동일한 클러스터 내의 여러 세션에서 공유될 수 있지만, 여전히 영구적이지는 않습니다.

3.7.3 뷰 (Views) / 가상 테이블 (Virtual Tables)

- **한 줄 핵심 요약:** 쿼리 정의에 의해 생성되는 가상 테이블로, 실제 데이터를 저장하지 않고 참조될 때마다 쿼리를 실행하여 결과를 생성합니다.
- **직관적 비유:** 특정 보고서를 만들 때 필요한 데이터를 매번 여러 테이블에서 가져와 조합하는 대신, 그 조합 과정을 미리 정의해둔 '레시피'와 같습니다. 레시피 자체는 음식이 아니지만, 레시피를 사용하면 항상 신선한 음식을 만들 수 있습니다.
- **기술적 설명:**
 - **정의:** 뷰는 쿼리(Query)에 의해 정의됩니다.
 - **동작:** 뷰를 참조할 때마다 기본 쿼리가 실행되어 최신 결과를 반환합니다.

3.7.4 구체화된 뷰 (Materialized Views)

- **한 줄 핵심 요약:** 뷰와 유사하지만, 쿼리 결과를 미리 계산하여 실제 데이터로 저장해두는 테이블입니다.
- **직관적 비유:** 자주 만드는 보고서의 결과(음식)를 미리 만들어 냉장고에 보관해두는 것과 같습니다. 필요할 때 즉시 꺼내 쓸 수 있어 빠르지만, 시간이 지나면 신선도가 떨어질 수 있습니다.
- **기술적 설명:**
 - **장점:** 쿼리 결과를 캐시(Cache)하여 성능을 크게 향상시킵니다. 집계(Aggregation) 함수나 조인(Join)이 많은 BI(Business Intelligence) 보고서에 유용합니다.
 - **단점:** 기본 데이터가 변경되어도 구체화된 뷰는 자동으로 업데이트되지 않을 수 있으므로, 주기적인 새로 고침(Refresh)이 필요합니다. 그렇지 않으면 오래된(Stale) 데이터를 반환할 수 있습니다.

3.7.5 스트리밍 테이블 (Streaming Tables)

- **한 줄 핵심 요약:** 스트리밍 파이프라인을 위해 특별히 설계된 테이블로, 새로운 이벤트나 데이터가 들어올 때마다 자동으로 업데이트됩니다.
- **직관적 비유:** 주식 시장의 실시간 시세판처럼, 새로운 거래 정보가 들어올 때마다 화면이 자동으로 갱신되어 항상 최신 정보를 보여주는 것과 같습니다.

- 기술적 설명:

- 특징: 스트리밍 파이프라인의 일부로, 데이터가 지속적으로 유입됨에 따라 테이블이 자동으로 최신 상태를 유지합니다.

3.8 관리형 테이블 (Managed Tables) vs 외부 테이블 (External Tables)

Databricks와 같은 플랫폼에서는 데이터와 메타데이터를 관리하는 방식에 따라 테이블을 두 가지로 분류합니다.

Table 2: 관리형 테이블 vs 외부 테이블 비교

특징	관리형 테이블 (Managed Tables)	외부 테이블 (External Tables)
데이터 위치	웨어하우스 경로 내에서 플랫폼이 관리	Databricks 외부에서 생성 및 관리
데이터/메타데이터 관리	데이터베이스(플랫폼)가 모두 관리	데이터베이스는 메타데이터만 관리, 데이터는 외부 파일 시스템에 존재
테이블 삭제 시 동작	메타데이터와 실제 데이터 모두 삭제	메타데이터만 삭제, 외부 파일은 유지
공유 용이성	상대적으로 공유하기 어려움	외부 파일이므로 공유하기 쉬움
데이터 손실 위험	테이블 삭제 시 데이터도 손실되므로 높음	테이블 삭제 시 데이터는 유지되므로 낮음
생성 코드	형식(Format)과 위치(Location)를 지정하지 않으면 관리형으로 간주 (더 간단)	형식과 위치를 명시적으로 지정해야 함

3.9 데이터 파이프라인 개요

데이터 파이프라인은 데이터 통합 및 엔지니어링의 핵심 요소로, 데이터를 한 지점에서 다른 지점으로 이동시키거나, 여러 소스에서 수집, 변환하여 다양한 목적지로 저장하는 과정을 자동화하는 시스템입니다.

3.9.1 정의 및 역할

- **한 줄 핵심 요약:** 데이터를 소스에서 목적지까지 이동, 변환, 저장하는 일련의 자동화된 과정입니다.
- **직관적 비유:** 공장에서 원자재(데이터)를 들여와 여러 공정(변환)을 거쳐 완제품(처리된 데이터)을 만들고, 이를 최종 소비자(분석 시스템, 애플리케이션)에게 전달하는 컨베이어 벨트 시스템과 같습니다.
- **기술적 설명:**
 - **단순한 형태:** 데이터를 A 지점에서 B 지점으로 단순히 이동시키는 것일 수 있습니다.
 - **복잡한 형태:** 여러 데이터 소스에서 데이터를 수집(Ingest)하고, 여러 변환(Transformation)을 거쳐, 여러 출력 위치(Output Locations) 또는 목적지(Destinations)에 데이터를 저장하는 복잡한 과정일 수 있습니다.
 - **ETL 파이프라인:** 데이터 파이프라인의 특별한 경우로, 일반적으로 데이터를 수집, 변환한 후 분석 또는 애플리케이션, 머신러닝/AI 시스템에서 소비할 수 있는 형태로 출력합니다. 비즈니스 요구 사항과 의사 결정을 지원하도록 설계됩니다.

Figure 1: 데이터 파이프라인과 ETL 파이프라인의 관계 (벤 다이어그램)

위 그림은 데이터 파이프라인이 더 넓은 개념이며, ETL 파이프라인은 데이터 파이프라인의 특정 유형임을 보여주는 벤 다이어그램을 나타냅니다.

3.9.2 데이터 파이프라인의 생명 주기

데이터 파이프라인은 소프트웨어 애플리케이션과 유사하게 관리됩니다.

1. **설계 (Design):** 비즈니스 요구 사항과 데이터 흐름을 정의합니다.

2. **구현 (Implement):** 정의된 설계에 따라 코드를 작성하고 시스템을 구축합니다.
3. **테스트 (Test):** 파이프라인이 예상대로 작동하고 데이터가 정확하게 처리되는지 검증합니다.
4. **배포 (Deploy):** 운영 환경에 파이프라인을 배포합니다.
5. **모니터링 (Monitor):** 파이프라인의 성능, 오류, 데이터 품질 등을 지속적으로 감시합니다.
6. **변경 처리 및 업데이트 (Handle Changes and Update):** 데이터 소스, 스키마, 비즈니스 로직 등의 변경 사항에 대응하여 파이프라인을 업데이트하고 재배포합니다.

3.9.3 자동화 및 트리거

- **자동화 (Automation):** 파이프라인이 수동 개입 없이 자체적으로 실행될 수 있도록 자동화하는 것이 중요합니다. 이는 주요 장점 중 하나입니다.
- **트리거 (Triggers):** 파이프라인이 언제 실행될지 결정하는 요소입니다.
 - **파일 기반 (File-based):** 파일 시스템에 새로운 파일이 나타날 때.
 - **스케줄 기반 (Schedule-based):** Cron 스케줄과 같이 정해진 시간에 주기적으로 실행.
 - **수동 (Manual):** 사용자가 직접 실행.

3.9.4 데이터 파이프라인의 장점

- **IT 팀 부하 감소:** 자동화를 통해 IT 팀의 수동 작업을 줄여줍니다.
- **데이터 소비 및 전달 자동화:** 사용자에게 데이터가 더 빠르고 효율적으로 전달됩니다.
- **클라우드 기반 동적 자원 활용:** 데이터 볼륨이 증가할 때 클라우드의 탄력적인 자원(Elasticity)을 활용하여 부하를 처리할 수 있습니다.
- **실시간 분석 및 의사 결정 지원:** 비즈니스에 중요한 실시간 분석 및 의사 결정을 가능하게 합니다.

3.9.5 데이터 파이프라인의 과제

- **데이터 문제:** 스키마 변경, 데이터 품질 저하 등 데이터 자체의 문제가 파이프라인 오류를 유발할 수 있습니다. 이러한 문제를 해결하는 데 시간이 걸리고 서비스 중단(Outages)을 초래할 수 있습니다.
- **프레임워크 의존성:** 파이프라인이 구축된 프레임워크에 강하게 의존하는 경향이 있습니다. 프레임워크 변경은 파이프라인에 문제를 일으킬 수 있습니다.

3.10 데이터 파이프라인 유형

테이블 유형과 마찬가지로, 데이터 파이프라인도 다양한 처리 방식과 목적에 따라 여러 유형으로 나뉩니다.

3.10.1 배치 파이프라인 (Batch Pipelines)

- **한 줄 핵심 요약:** 일정량의 데이터를 모아 한 번에 처리하는 방식입니다.
- **직관적 비유:** 세탁물을 모아서 한 번에 세탁하는 것과 같습니다. 일정량이 모일 때까지 기다렸다가 처리하므로 효율적이지만, 즉각적인 처리는 어렵습니다. **사용 사례:** 보고서 생성, 청구서 처리, 과거 데이터 분석(Historical Analysis) 등.

3.10.2 스트리밍 파이프라인 (Streaming Pipelines) / 실시간 파이프라인 (Real-time Pipelines)

- **한 줄 핵심 요약:** 데이터가 생성되는 즉시 지속적으로 수집하고 처리하는 방식입니다.
- **직관적 비유:** 수도꼭지에서 물이 나오는 즉시 컵에 받아 마시는 것과 같습니다. 데이터가 들어오는 대로 즉시 처리하므로 최신 정보를 얻을 수 있습니다.
- **사용 사례:** 사기 탐지 (Fraud Detection), 실시간 대시보드, 추천 시스템, IoT(사물 인터넷) 장치 이벤트 처리 등.

3.10.3 ETL 및 ELT 파이프라인

- **ETL (Extract, Transform, Load):** 전통적인 배치 지향 처리 방식으로, 데이터를 추출, 변환한 후 적재합니다. 들어오는 데이터의 스키마가 잘 알려져 있을 때 적합합니다.
- **ELT (Extract, Load, Transform):** 데이터를 추출, 적재한 후 변환합니다. 들어오는 데이터의 스키마가 덜 알려져 있거나 유연한 분석이 필요할 때 적합합니다.

3.10.4 기타 파이프라인 유형

- **데이터 복제 (Data Replication):** 데이터를 다른 위치로 복사하여 백업 또는 고가용성을 제공합니다.
- **머신러닝 파이프라인 (Machine Learning Pipelines):** 머신러닝 모델 생성 과정을 포함합니다. (추후 강의에서 다룰 예정)
- **워크플로우 및 오케스트레이션 파이프라인 (Workflow and Orchestration Pipelines):** 데이터 소스와 목적지를 변경할 수 있도록 더 유연하고 구성 가능한 파이프라인입니다.

3.11 스트리밍 데이터 파이프라인의 필요성 및 활용

스트리밍 데이터 파이프라인은 데이터가 생성되는 즉시 처리하여 실시간 인사이트를 제공하고, 빠른 의사 결정을 지원하는 데 필수적입니다.

3.11.1 왜 스트리밍 데이터 파이프라인이 중요한가?

- **속도 (Speed):** 모든 것은 속도에 관한 것입니다. 들어오는 데이터를 가능한 한 빨리 분석하여 인사이트를 제공해야 합니다.
- **실시간 처리 (Real-time Processing):** 데이터 수집, 분석, 저장에 모두 실시간으로 이루어져야 합니다.
- **대규모 데이터 처리 (Handling Large Amounts of Data):** 대량의 데이터를 효율적으로 처리할 수 있어야 합니다.
- **시계열 데이터 (Time Series Data):** 타임스탬프가 찍히고 시간 순서로 정렬되어 들어오는 데이터에 특히 잘 작동합니다.

3.11.2 스트리밍이 필요한 상황

- **센서 데이터 (Sensor Data):** IoT 장치에서 실시간으로 수집되는 온도, 습도, 위치 정보 등.
- **광고 서빙 (Ad Serving):** 사용자 행동에 기반한 실시간 광고 추천.
- **보안 애플리케이션 (Security Applications):** 이상 징후 탐지, 침입 감지.

- **클릭스트림 데이터 (Clickstream Data):** 웹사이트 사용자 행동 분석.
- **결제 거래 (Payment Transactions):** 실시간 사기 탐지.

3.11.3 배치 작업을 스트리밍으로 구현하는 방법

- 흥미롭게도, 배치 작업도 스트리밍 파이프라인으로 구현할 수 있습니다.
- **원리:** 스트리밍 파이프라인의 트리거 (Trigger) 주기를 길게 설정 (예: 24시간마다 한 번) 하면, 이는 사실상 배치 처리와 동일하게 작동합니다.
- **장점:** 스트리밍 프레임워크가 제공하는 강력한 기능 (예: 체크포인트, 오류 복구, exactly-once 처리)을 배치 작업에도 활용할 수 있습니다.

3.11.4 비즈니스 요구 사항에 집중

- 데이터 파이프라인을 설계할 때 가장 중요한 것은 IT 부서에 편리한 것보다 비즈니스 요구 사항에 집중하는 것입니다.
- **고려 사항:**
 - 데이터 접근 속도 (Speed of Access)
 - 데이터 정확성 (Accuracy of Data)
 - 어떤 데이터가 필요한가?
 - 비즈니스가 어떤 질문에 대한 답변을 필요로 하는가?

3.12 스트리밍 핵심 개념

스트리밍 데이터 파이프라인을 이해하고 설계하는 데 필요한 주요 개념들입니다.

- **소스 (Source), 처리 (Processing), 싱크 (Sink):**
 - 모든 데이터 파이프라인은 데이터를 가져오는 소스, 데이터를 변환하거나 분석하는 처리 구성 요소, 그리고 처리된 데이터를 쓰는 싱크로 구성됩니다.
- **파일 기반 스트리밍 (File-based Streaming) vs 이벤트 기반 스트리밍 (Event-based Streaming):**
 - **파일 기반:** 데이터가 파일에서 읽히거나 파일로 쓰여집니다. (예: 새로운 파일이 특정 디렉토리에 도착하면 처리)
 - **이벤트 기반:** 데이터가 이벤트 형태로 소비되거나 생성됩니다. (예: 메시지 큐에서 이벤트를 읽거나 이벤트를 발행)
- **마이크로 배치 (Microbatch) vs 연속 스트리밍 (Continuous Streaming):**
 - **마이크로 배치:** 스트리밍 데이터를 작은 덩어리 (배치)로 나누어 주기적으로 처리합니다. (예: Spark Structured Streaming의 기본 모드)
 - **연속 스트리밍:** 들어오는 스트림을 끊임없이 처리합니다. (예: Flink)
- **처리 트리거 (Processing Trigger):**
 - 마이크로 배치를 시작하는 데 사용되는 메커니즘입니다. (예: 1초마다, 5초마다)
- **출력 모드 (Output Modes):**
 - 스트림에 데이터를 쓸 때 데이터를 처리하는 방식입니다.
 - **Append (추가):** 새로운 데이터만 추가합니다.
 - **Overwrite (덮어쓰기):** 기존 데이터를 모두 삭제하고 새로운 데이터로 덮어씁니다.

- **Update (업데이트):** 기존 데이터를 업데이트하거나 새로운 데이터를 추가합니다.
- **체크포인트 (Checkpoint):**
 - 데이터가 올바르게 기록된 마지막 지점을 기록합니다.
 - 장애 발생 시 체크포인트 지점으로 돌아가 복구하고, 그 이후의 데이터만 재처리하여 데이터 손실이나 중복을 방지합니다.
- **윈도우 (Window):**
 - 집계(Aggregation) 함수(예: 합계, 개수, 평균)에 사용되는 시간 간격입니다.
 - 특정 시간 범위 내의 데이터를 그룹화하여 집계합니다. (예: 지난 5분간의 평균 온도)
- **워터마크 (Watermark):**
 - 지연 도착 데이터(Late Arriving Data)를 처리하기 위한 기준 시간입니다.
 - 워터마크를 넘어 너무 늦게 도착한 데이터는 오래된(Stale) 것으로 간주되어 무시될 수 있습니다. 워터마크 내의 데이터는 허용되고 처리됩니다.
- **스트림 작업 (Stream Operations):**
 - 스트리밍 파이프라인 내에서 발생하는 모든 작업(필터링, 변환, 집계 등)을 의미합니다.
- **인스트림 분석 (In-stream Analytics):**
 - 스트림에서 데이터를 읽어 즉시 처리하며, 데이터를 디스크나 파일에 쓰지 않습니다.
 - 데이터를 디스크에 쓰고 다시 읽는 과정을 생략하므로 더 빠릅니다. 주로 메모리 내에서 처리됩니다.

3.13 데이터 처리 속도 스펙트럼

모든 사용 사례가 실시간 처리를 요구하는 것은 아니며, 비즈니스 요구 사항에 따라 적절한 처리 속도를 선택하는 것이 중요합니다.

Table 3: 데이터 처리 속도 스펙트럼 및 사용 사례

처리 속도	특징	사용 사례
배치 (Batch)	데이터를 모아 주기적으로 한 번에 처리	계량 및 청구, 임시 분석, 비즈니스 인텔리전스 (BI)
준실시간 (Near Real-time)	데이터를 몇 분 이내에 처리하여 사용 가능	모바일/IoT 데이터 처리, 로그 수집, 광고 서빙, 클릭스트림 분석
실시간 (Real-time)	데이터를 몇 초 또는 1초 미만에 처리하여 즉시 반응	신용카드 사기 탐지, 금융 자산 거래, 멀티플레이어 게임, ML 추론, 환자 모니터링

주의사항

중요: 모든 사용 사례에 실시간 처리가 필요한 것은 아닙니다. 불필요하게 실시간 시스템을 구축하면 비용과 복잡성이 증가하므로, 사용 사례에 맞는 적절한 처리 속도를 선택해야 합니다.

3.14 Lambda vs Kappa 아키텍처

데이터 파이프라인을 설계하는 두 가지 주요 아키텍처 패턴입니다.

3.14.1 Lambda 아키텍처 (Lambda Architecture)

- **한 줄 핵심 요약:** 배치 처리와 스트리밍 처리를 병렬로 수행하는 두 개의 분리된 레이어를 사용하여 데이터의 속도와 신뢰성을 모두 확보하려는 아키텍처입니다.
- **직관적 비유:** 중요한 문서를 보관할 때, 원본 문서는 안전한 금고(배치 레이어)에 보관하고, 급하게 필요한 정보는 복사본을 만들어 빠르게 전달(스트리밍 레이어)하는 것과 같습니다. 두 정보는 나중에 합쳐져 최종 보고서가 됩니다.

- 기술적 설명:

- 구성:

1. **배치 레이어 (Batch Layer):** 대량의 데이터를 배치 모드로 처리하여 정확하고 신뢰성 있는 결과를 생성합니다. (더 신뢰할 수 있다고 간주됨)
2. **스트리밍 레이어 (Streaming Layer) / 속도 레이어 (Speed Layer):** 들어오는 데이터를 실시간 또는 준실시간으로 처리하여 빠른 결과를 제공합니다.
3. **서빙 레이어 (Serving Layer):** 배치 레이어와 스트리밍 레이어의 결과를 결합하여 사용자에게 제공합니다.

- **장점:** 배치 처리의 신뢰성과 스트리밍 처리의 속도를 모두 활용할 수 있습니다.

- 단점:

- * **데이터 처리 중복:** 동일한 데이터를 두 가지 방식으로 처리해야 하므로 복잡성이 증가합니다.
- * **결과 일치 문제:** 배치 레이어와 스트리밍 레이어의 결과를 결합하고 일치시키는 과정 (Reconciliation) 이 어렵습니다.
- * **유지보수:** 두 개의 독립적인 코드베이스와 시스템을 유지보수해야 합니다.

3.14.2 Kappa 아키텍처 (Kappa Architecture)

- **한 줄 핵심 요약:** 모든 데이터를 스트리밍으로 처리하는 단일 레이어를 사용하여 Lambda 아키텍처의 복잡성을 줄이고 시스템을 단순화한 아키텍처입니다.
- **직관적 비유:** 모든 문서를 디지털화하여 하나의 클라우드 시스템에 저장하고, 필요할 때마다 이 시스템에서 실시간으로 정보를 검색하거나 보고서를 생성하는 것과 같습니다. 원본과 복사본을 따로 관리할 필요가 없습니다.

- 기술적 설명:

- **구성:** 단일 스트리밍 레이어 (Single Streaming Layer) 만 존재합니다. 이 스트리밍 레이어가 마이크로 배치 (Microbatches) 또는 연속 스트리밍 방식으로 배치 처리와 스트리밍 처리를 모두 지원합니다.

- **장점:**

- * **단순성:** 하나의 시스템만 유지보수하면 되므로 아키텍처가 훨씬 단순합니다.
- * **확장성 (Scalability):** 단일 시스템이므로 확장성이 더 좋습니다.
- * **일관성 (Consistency):** 모든 데이터가 동일한 스트리밍 레이어를 통해 처리되므로 데이터 일관성을 유지하기 쉽습니다. 특히 **Exactly-Once Processing** (정확히 한 번 처리) 을 지원하여 동일한 데이터가 두 번 처리되는 문제를 방지합니다.

- **단점:** 스트리밍 기술의 성숙도와 강력한 스트리밍 엔진이 필요합니다.

Table 4: Lambda vs Kappa 아키텍처 비교

특징	Lambda 아키텍처	Kappa 아키텍처
레이어 수	배치 레이어 + 스트리밍 레이어 (2개)	스트리밍 레이어 (1개)
처리 방식	배치 (정확성) + 스트리밍 (속도)	모든 데이터를 스트리밍으로 처리 (배치도 스트리밍의 일종으로 간주)
복잡성	높음 (두 개의 시스템, 데이터 중복, 결과 일치 문제)	낮음 (단일 시스템, 단일 데이터 흐름)
데이터 일관성	배치/스트리밍 결과 일치 문제 발생 가능	Exactly-Once 처리로 높은 일관성 유지
유지보수	두 개의 코드베이스 관리 필요	하나의 코드베이스 관리
등장 시기	초기 스트리밍 기술이 미성숙했을 때 등장	스트리밍 기술이 성숙한 이후 등장
선호도	현재는 Kappa 아키텍처가 더 선호됨	현대적이고 단순하여 더 선호됨

주의사항

AWS Lambda와 혼동 주의: AWS Lambda는 서버리스 컴퓨팅 서비스의 이름이며, 여기서 다루는 Lambda 아키텍처와는 다른 개념입니다. Lambda 및 Kappa는 특정 제품이 아닌 데이터 파이프라인 설계 아키텍처 패턴을 지칭하는 산업 용어입니다.

3.15 스트리밍 처리 프레임워크

다양한 스트리밍 처리 프레임워크가 있으며, 각각 고유한 특징과 사용 사례를 가집니다.

Table 5: 주요 스트리밍 처리 프레임워크 비교

프레임워크	처리 모델	지연 시간	내결합성	사용 편의성	주요 특징/사용 사례
Kafka Streams	이벤트 기반	낮음	높음 (RocksDB 상태 관리)	중간 (Java 라이브러리)	Kafka 생태계 통합, 상태 저장 처리
Apache Flink	이벤트 기반 (진정한 스트리밍)	매우 낮음	Exactly-Once	어려움 (고급 시스템)	복잡 이벤트 처리 (CEP), 조저지연
Spark Structured Streaming	마이크로 배치 (연속 모드 지원)	중간 (수백 ms)	Exactly-Once (체크포인트)	매우 좋음 (통합 API)	배치/스트리밍 통합, Lakehouse 기반
Apache Storm	이벤트 기반	매우 낮음	At-Least-Once	어려움 (레거시)	초기 스트리밍 프레임워크, 현재는 레거시
Apache Samza	이벤트 기반	낮음	높음	중간	LinkedIn 개발, YARN/Kafka 통합
KSQL DB	이벤트 기반 (Kafka SQL)	낮음	높음 (Kafka 상속)	매우 좋음 (SQL)	Kafka 위에 SQL 레이어, 연속 SQL 쿼리
AWS Kinesis	이벤트 기반	낮음	높음	좋음	AWS 클라우드 네이티브, 고성능 스트리밍
Google Cloud Dataflow (Beam)	이벤트 기반	낮음	높음	좋음	Google 클라우드 네이티브, Apache Beam 기반
Azure Stream Analytics	이벤트 기반	낮음	높음	좋음	Azure 클라우드 네이티브

3.15.1 파일 기반 vs 이벤트 기반 스트리밍

- **파일 기반 스트리밍 (File-based Streaming):**
 - 데이터가 디스크에 먼저 기록된 후, 다른 프로세스가 이를 읽어 처리하는 방식입니다.
 - 특징: 디스크 I/O가 발생하므로 느립니다.
- **이벤트 기반 스트리밍 (Event-based Streaming):**
 - 데이터가 생성되는 즉시 호스트에서 직접 가져와 처리하는 방식입니다.
 - 특징: 디스크 I/O를 최소화하므로 빠릅니다.
 - **1세대 (메시지 브로커):** ActiveMQ, RabbitMQ와 같은 메시지 지향 미들웨어 (Message Oriented Middleware)를 사용했습니다.
 - **2세대 (분산 스트리밍 플랫폼):** Kafka, Kinesis와 같은 플랫폼이 사실상의 표준이 되었습니다. 이들은 고성능, 영속성, 대용량 트래픽 처리를 지원하며 스트리밍에 특화되어 있습니다.

3.15.2 Kafka의 주요 개념

- **프로듀서 (Producer):** 데이터를 Kafka 클러스터로 보내는 주체입니다.
- **컨슈머 (Consumer):** Kafka 클러스터에서 데이터를 읽어가는 주체입니다.
- **토픽 (Topic):** 데이터 스트림을 분류하는 논리적인 채널입니다. 프로듀서는 특정 토픽에 데이터를 쓰고, 컨슈머는 특정 토픽을 구독하여 데이터를 읽습니다.
- **파티션 (Partition):** 토픽은 여러 파티션으로 나뉘어 데이터를 병렬로 처리하고 확장성을 높입니다.

3.16 Databricks 엔드투엔드 스트리밍 아키텍처

Databricks는 클라우드 기반의 통합 데이터 및 AI 플랫폼으로, 다양한 데이터 소스부터 최종 애플리케이션까지의 엔드투엔드 데이터 흐름을 지원합니다.

위 그림은 *Databricks* 플랫폼을 중심으로 한 데이터 흐름을 보여줍니다. 다양한 데이터 소스에서 데이

Figure 2: Databricks 기반 엔드투엔드 스트리밍 아키텍처 (개념도)

터가 수집되어 클라우드 스토리지 또는 메시지 버스에 랜딩되고, *Delta Lake*, *Unity Catalog*, *Photon*, *Structured Streaming*을 통해 처리된 후, *Databricks SQL*, *ML*, *BI/AI* 앱으로 소비되는 과정을 나타냅니다.

3.16.1 데이터 소스 (Data Sources)

- 데이터 웨어하우스, 온프레미스 시스템, SaaS 애플리케이션, 머신/애플리케이션 로그, 이벤트, IoT 데이터 등 다양한 소스에서 데이터가 발생합니다.
- Databricks는 클라우드 전용 플랫폼이지만, 온프레미스 시스템의 데이터를 클라우드로 마이그레이션하는 추세가 강합니다.

3.16.2 데이터 랜딩 (Data Landing)

- **파일 스토리지 (File Storage):** 클라우드 기반 스토리지 (예: AWS S3, Azure Data Lake Storage (ADLS))에 파일 형태로 저장됩니다.
- **메시지 버스 (Message Buses):** Kafka, Kinesis, Event Hubs와 같은 분산 메시지 시스템을 통해 실시간 이벤트 스트림이 수집됩니다.

3.16.3 데이터 처리 계층 (Data Processing Layer)

- **Delta Lake:** 클라우드 스토리지에 저장된 데이터에 ACID 트랜잭션, 스키마 진화, 시간 여행 등의 기능을 추가하여 데이터의 신뢰성과 분석 준비 상태를 보장합니다. 데이터는 여전히 오픈 포맷으로 저장됩니다.
- **Unity Catalog:** 메타데이터 관리, 데이터 거버넌스, 접근 제어, 데이터 계보(Lineage), 감사 로그(Audit Logs) 등을 제공하는 통합 카탈로그입니다. 행/컬럼 수준 보안도 지원합니다.
- **Photon:** Spark 엔진을 C++로 재작성한 고성능 쿼리 엔진입니다. JVM 기반 Spark의 비효율성을 개선하여 더 빠른 데이터 처리와 벡터화된 연산을 제공합니다.
- **Structured Streaming / DLT (Lakehouse Declarative Pipelines, LDP):** Apache Spark 기반의 스트리밍 처리 API로, 배치 및 스트리밍 처리를 통합합니다. DLT(이전 명칭)는 Structured Streaming 위에 구축된 선언적 파이프라인 프레임워크입니다. 이 모든 것은 오픈 소스 기반입니다.

3.16.4 데이터 소비 (Data Consumption)

- **Databricks SQL:** 쿼레이션된 데이터에 대해 실시간 분석, 대시보드, 보고서를 생성할 수 있는 SQL 인터페이스를 제공합니다. NoSQL 데이터에 대해서도 SQL 쿼리를 실행할 수 있습니다.
- **머신러닝 (Machine Learning):** Databricks 플랫폼은 ML 런타임(Runtime)을 제공하여 일반적인 ML 패키지들이 사전 설치 및 검증되어 있습니다. 데이터 과학자들이 종속성 문제 없이 쉽게 ML 모델을 개발하고 배포할 수 있도록 돕습니다.
- **엔터프라이즈 애플리케이션 (Enterprise Applications):** Tableau, PowerBI, Looker와 같은 BI 도구와 통합될 수 있으며, 실시간 AI 앱이나 운영 앱(예: 사기 패턴 감지 시 알림)과도 연동됩니다.

3.17 Databricks Jobs 기능

Databricks Jobs는 데이터 파이프라인의 워크플로우 오케스트레이션, 안정성 및 관찰 가능성을 위한 다양한 기능을 제공합니다.

- **파일 도착 트리거 (File Arrival Triggers):** 새로운 파일이 도착하면 파이프라인을 자동으로 시작합니다. 파일 도착 시간을 예측하기 어려울 때 컴퓨팅 자원 낭비를 줄일 수 있습니다.
- **조건부 작업 (Conditional Tasks):** 워크플로우 내에서 이전 작업의 결과에 따라 다른 경로를 선택할 수 있습니다. (예: 이전 작업이 성공하면 A 경로, 실패하면 B 경로)
- **서버리스 컴퓨팅 (Serverless Compute):** 컴퓨팅 자원을 직접 관리할 필요 없이, Databricks가 자동으로 자원을 프로비저닝하고 스케일링합니다. (무료 에디션에서 사용 가능)
- **작업 수준 매개변수 (Job Level Parameters):** 작업에 매개변수를 전달하여 동일한 코드를 다른 소스나 목적지에 대해 재사용할 수 있습니다. (예: 500개의 파이프라인 대신 하나의 파이프라인에 500개의 매개변수)
- **웹훅 (Webhooks):** 파이프라인 상태 변경(성공, 실패 등) 시 Slack, 이메일, PagerDuty 등으로 알림을 보냅니다.
- **작업 큐 (Job Queues):** 동시 실행 가능한 작업 수를 제한하거나, 새로운 트리거를 큐에 넣어 순차적으로 처리합니다.
- **연속 실행 (Continuous Execution) 및 작업 루핑 (Task Looping):** 마이크로 배치 또는 연속 모드로 파이프라인을 지속적으로 실행하거나, 특정 작업을 여러 번 반복 실행할 수 있습니다.
- **테이블 트리거 (Table Triggers):** 특정 테이블에 새로운 데이터가 채워지면 다른 파이프라인을 트리거합니다.
- **시각적 모니터링 (Visual Monitoring):** 파이프라인의 실행 상태, 소요 시간, 실패 여부 등을 대시보드에서 시각적으로 확인할 수 있습니다.
- **시스템 테이블의 작업 데이터 (Jobs Data in System Tables):** 작업 실행에 대한 메타데이터(실행 시간, ID, 트리거 사용자, 소요 시간 등)가 시스템 테이블에 저장되어 나중에 분석할 수 있습니다. (예: 어제 작업이 1시간 걸렸는데 평소에는 30분 걸렸다면 원인 분석)
- **모듈형 워크플로우 (Modular Workflows):** 다른 작업을 호출하거나 다른 워크플로우를 포함하여 모듈화된 파이프라인을 구성할 수 있습니다.
- **실행 시간 알림 (Duration Alerts):** 작업 실행 시간이 예상보다 길어지면 알림을 보내거나 작업을 중단하여 불필요한 컴퓨팅 비용을 방지합니다.
- **간트 시각화 (Gantt Visualization):** 작업의 진행 상황과 소요 시간을 간트 차트 형태로 시각적으로 보여줍니다.
- **SQL 작업 (SQL Tasks):** SQL 쿼리를 직접 작업으로 실행할 수 있습니다.
- **Databricks Asset Bundles (DAB):** CI/CD(지속적 통합/지속적 배포)를 지원하여 코드 프로모션(개발 환경에서 운영 환경으로)을 자동화합니다. (추후 강의에서 다룰 예정)

이러한 기능들은 데이터 엔지니어의 작업을 훨씬 효율적이고 안정적으로 만들어줍니다.

3.18 스트리밍 파이프라인 설계 고려사항 및 트레이드오프

스트리밍 파이프라인을 설계할 때는 다양한 요구 사항과 제약 사항 사이에서 균형을 찾아야 합니다. Spark는 성능, 비용, 품질을 조절할 수 있는 유연성을 제공하지만, 모든 것을 동시에 얻을 수는 없습니다 (CAP

이론의 변형).

3.18.1 반복적인 설계 과정

1. **목표 및 요구 사항 이해:** 스트리밍 사용 사례의 목표와 요구 사항(예: 데이터의 중요성)을 명확히 이해합니다.
2. **전략 수립:** 각 목표를 달성하기 위한 전략을 세웁니다.
3. **자원 정의:** 각 목표에 필요한 컴퓨팅, 스토리지 등의 자원을 정의합니다.
4. **비용 계산:** 필요한 자원의 비용을 계산합니다.
5. **트레이드오프 재검토:** 기대치와 실제 비용/성능 사이의 균형을 맞추기 위해 트레이드오프를 재검토합니다.

3.18.2 주요 고려 사항

- **확장성 (Scalability):**
 - **데이터 볼륨 (Data Volume):** 얼마나 많은 데이터가 유입될 것인가?
 - **TPS (Throughput Per Second):** 초당 처리량은 얼마인가? 높은 처리량을 위해서는 그에 맞는 컴퓨팅 자원 설계가 필요합니다.
- **성능 (Performance):**
 - **종단 간 지연 시간 (End-to-End Latency):** 데이터가 들어와서 처리되고 결과가 나올 때까지의 시간은 얼마인가?
- **처리 요구 사항 (Processing Needs):**
 - **데이터 정제 수준:** 데이터가 얼마나 깨끗한가? 정제, 변환, 정규화, 큐레이션 등 얼마나 많은 작업이 필요한가?
 - **작업 복잡성:** 몇 번의 홉(Hops), 싱크(Sinks), 조인(Joins)이 필요한가? 상태 저장 처리(Stateful Processing)가 필요한가? 재처리 로직(Reprocessing Logic)이 필요한가?
- **품질 (Quality):**
 - **데이터 정확성:** 데이터가 올바른가? 데이터 유실(No Drops)이나 중복(No Duplicates)이 없는가?
 - **멱등성 (Idempotency):** 동일한 파이프라인을 여러 번 실행해도 결과가 동일한가? (중복 파일 처리 방지)
- **신뢰성 및 가용성 (Reliability and Availability):**
 - **장애 복구 (Failure Recovery):** 장애 발생 시 어떻게 복구할 것인가?
 - **재해 복구 (Disaster Recovery, DR):** 전체 리전(Region)이 다운될 경우 어떻게 대응할 것인가? (매우 복잡한 사용 사례)

3.18.3 자원 고려 사항

- **컴퓨팅 (Compute):**
 - 필요한 컴퓨팅 자원의 양과 유형 (VM 수, 유형, 풀). 서버리스 환경에서는 이 부분이 추상화되지만, 직접 클러스터를 관리할 때는 중요합니다.
- **스토리지 (Storage):**
 - 데이터를 어디에 저장할 것인가? (웜 스토리지, 콜드 스토리지). 추가 서비스 통합, API 호출 및 요금 제한.

- **인력 (People):**

- 파이프라인 개발, 관리, 유지보수에 필요한 인력의 기술 수준.

3.18.4 실제 시나리오: 네트워크 위협 탐지 시스템 비용 문제

- **문제:** 한 대규모 회사에서 네트워크 위협 탐지 시스템을 Databricks로 구축했습니다. 이벤트 허브에서 데이터를 수집하고 모델을 적용하여 위협 패턴을 예측했습니다. 하지만 스토리지 비용이 70%, VM(컴퓨팅) 비용이 30%로 비정상적으로 높게 나왔습니다. 일반적으로 컴퓨팅 비용이 더 높아야 합니다.
- **원인 분석:**
 - **액세스 계층 (Access Tier):** 활성 데이터(Active Data)를 워밍 스토리지(Warm Storage)에 보관해야 하는데, 자주 접근하는 데이터를 쿨드 스토리지(Cool Storage)나 아카이브 스토리지(Glacier)에 보관하여 접근 비용이 매우 높아졌습니다. 쿨드 스토리지에서 데이터를 자주 읽으면 비용이 급증합니다.
 - **복제 방식 (Replication):** 재해 복구를 위해 지리적 중복 스토리지(Geo-redundant Storage)를 사용했는데, 이는 로컬 중복 스토리지(Local Redundant Storage)보다 훨씬 비쌉니다. 비즈니스 중요도에 따라 신중하게 선택해야 합니다.
 - **높은 송신 비용 (High Egress Cost):** 데이터를 클라우드 외부로 전송할 때 발생하는 비용이 높았습니다.
 - **작은 파일 문제 (Small Files Problem):** 낮은 스트리밍 볼륨으로 인해 매우 작은 파일들이 많이 생성되었고, 이는 컴퓨팅 경로에서 높은 오버헤드를 유발했습니다. (컴팩션(Compaction) 필요)
- **교훈:** 스토리지 계층, 복제 전략, 파일 크기 관리 등 세부적인 비용 요소를 면밀히 검토해야 합니다.

3.18.5 데이터 재처리 (Reprocessing) 및 따라잡기 (Catch-up)

시스템 장애 등으로 인해 데이터 처리가 지연되었을 때, 누락된 데이터를 처리하는 방법입니다.

- **옵션 1: 대규모 클러스터 확장 (Scale Up a Large Cluster):**
 - 기존 클러스터를 확장하여 처리량을 늘리고, 스로틀(Throttle)을 조정하여 백로그(Backlog)를 처리합니다.
 - **단점:** 백로그 처리 시간이 허용될 때만 가능하며, 체크포인트에 영향을 줄 수 있습니다.
- **옵션 2: 두 번째 클러스터 사용 (Spin Up a Second Cluster):**
 - 새로운 클러스터를 시작하고, 오프셋(Offset)을 조정하여 누락된 데이터부터 처리하도록 합니다.
 - 기존 클러스터는 계속해서 이전 데이터를 처리하고, 두 클러스터가 데이터를 따라잡으면 이전 클러스터를 종료합니다.
 - **단점:** 오프셋을 수동으로 변경하는 것은 위험할 수 있습니다.

3.18.6 로직 변경 (Changing Logic)

비즈니스 로직이 변경될 때 파이프라인을 업데이트하는 방법입니다.

- **방법:** 시간 제약이 없다면, 별도의 작업을 시작하여 변경된 로직으로 데이터를 재계산하고 따라잡도록 합니다. 완료되면 기존 작업을 중단하고 싱크(Sink)를 새로운 작업으로 전환합니다.

3.18.7 멀티테넌시 (Multi-tenancy)

여러 고객 또는 테넌트(Tenant)의 데이터를 하나의 시스템에서 처리하는 방법입니다.

- **문제:** 온프레미스에서는 각 고객마다 별도의 컴퓨팅 자원을 할당했지만, 클라우드에서는 매우 비효율적입니다.
- **해결책:**
 - **파티셔닝 (Partitioning):** 고객별로 데이터를 파티셔닝하여 격리합니다.
 - **스키마/카탈로그 격리 (Schema/Catalog Isolation):** 고객별로 스키마나 카탈로그를 분리하여 논리적 격리를 제공합니다.
 - **권한 관리 (Grants):** Unity Catalog와 같은 도구를 사용하여 고객별로 데이터 접근 권한을 세분화합니다.

3.18.8 중복 제거 (Deduplication) 및 워터마킹 (Watermarking)

- **워터마킹:** 지연 도착 데이터(Late Arriving Data)를 처리하기 위한 유예 기간을 설정합니다. (예: 버스가 10분 더 기다려주는 것과 같음)
- **중복 제거:** 워터마크 내에서 중복 데이터를 제거합니다.

3.19 스트리밍 파이프라인 모범 사례

- **SLA 이해 (Understand SLAs):** 서비스 수준 협약(Service Level Agreements)을 명확히 이해하여 파이프라인이 충족해야 할 성능 및 가용성 기준을 파악합니다.
- **비용 추정 (Ballpark Cost Estimate):** 프로젝트가 중단될 정도로 비용이 높아지지 않도록 초기 단계에서 대략적인 비용을 추정합니다.
- **항상 체크포인트 사용 (Always Use Checkpoint):** 장애 발생 시 데이터 손실 없이 복구할 수 있도록 체크포인트를 활용합니다.
- **처리 트리거 간격 도입 (Introduce Process Trigger Interval):** 비즈니스 요구 사항에 따라 적절한 트리거 간격을 설정하여 컴퓨팅 자원을 최적화합니다.
- **쓰기 최적화 (Optimize Your Writes):** 데이터를 효율적으로 저장하고, 작은 파일 문제를 방지하기 위해 컴팩션(Compaction) 등을 고려합니다.
- **용량 계획 (Plan for Capacity):** 데이터 유입량에 따라 파이프라인이 확장될 수 있도록 미리 용량을 계획합니다. (예: 모든 데이터를 한곳에 모았다가(MUX) 나중에 고객 유형별로 분리(DEMUX)하는 전략)

4 실습: 네트워크 로그 분석 (Lab 02)

이번 실습에서는 Databricks 환경에서 실제 네트워크 로그 데이터를 분석하는 과정을 다룹니다. 목표는 비정형 로그 데이터를 구조화하고, 참조 데이터를 결합하여 의미 있는 인사이트를 도출하는 것입니다.

4.1 실습 목표 및 환경 설정

4.1.1 실습 목표

- Databricks에서 공개 데이터셋을 로드하고 탐색합니다.
- 정규 표현식(Regular Expression)을 사용하여 비정형 텍스트 로그를 구조화된 데이터로 파싱합니다.
- PySpark 및 SQL 컨텍스트에서 사용자 정의 함수(UDF)를 생성하고 활용합니다.
- 외부 참조 데이터를 가져와 로그 데이터와 조인(Join)하여 데이터를 풍부하게 만듭니다.
- 변환된 데이터를 Delta 테이블에 저장하고, Spark의 실행 계획(Explain Plan)을 분석합니다.

4.1.2 환경 설정

1. **Lab 02 노트북 다운로드:** Canvas에서 lab02 network analysis 파일을 다운로드합니다.
2. **Databricks Workspace로 импорт:** Databricks Workspace의 홈 폴더 또는 원하는 위치로 이동하여, 우클릭 또는 상단 메뉴의 'Import'를 통해 다운로드한 zip 파일을 업로드합니다.
3. **컴퓨팅 연결:** 노트북을 열고 서버리스 컴퓨팅(Serverless Compute)에 연결합니다.
4. **초기 설정 셀 실행:** 첫 번째 셀을 실행하여 카탈로그, 스키마(lab02), 볼륨을 생성합니다. 이 셀은 멍등성(Idempotent)을 가지므로 여러 번 실행해도 문제가 없습니다.
5. **카탈로그 확인:** 카탈로그 탐색기에서 CSI 103 카탈로그 아래에 Lab02 스키마가 생성되었는지 확인합니다. 초기에는 테이블이 없지만, 볼륨은 생성되어 있을 것입니다.

```

1 # 예시: 초기설정셀내용
2 # 이코드는 Databricks 환경에서카탈로그 , 스키마, 볼륨을설정합니다 .
3 # 실제노트북에서는더많은변수설정이포함될수있습니다 .
4
5 catalog_name = "CSI103" # 사용자카탈로그이름
6 schema_name = "lab02" # 생성할스키마이름
7
8 spark.sql(f"CREATE CATALOG IF NOT EXISTS {catalog_name}")
9 spark.sql(f"USE CATALOG {catalog_name}")
10 spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema_name}")
11 spark.sql(f"USE SCHEMA {schema_name}")
12
13 # 볼륨생성예시 ( )
14 spark.sql(f"CREATE VOLUME IF NOT EXISTS {schema_name}.input_data")
15 spark.sql(f"CREATE VOLUME IF NOT EXISTS {schema_name}.output_data")
16
17 print(f현재" 카탈로그: {spark.catalog.currentCatalog()}")
18 print(f현재" 스키마: {spark.catalog.currentDatabase()}")

```

4.2 데이터 로드 및 탐색

4.2.1 공개 데이터셋 탐색

Databricks는 다양한 샘플 데이터셋을 공개적으로 마운트(Mount)하여 제공합니다.

- `dbutils.fs.ls("/databricks-datasets/")` 명령을 사용하여 사용 가능한 데이터셋 목록을 확인합니다.
- 이번 실습에서는 `/databricks-datasets/log_files/` *Apache* .

```
1 # 공개데이터셋목록확인
2 dbutils.fs.ls("/databricks-datasets/")
3
4 # 사용할로그파일경로정의및탐색
5 log_files_path = "/databricks-datasets/log_files/"
6 dbutils.fs.ls(log_files_path)
```

4.2.2 로그 파일 내용 확인

- `dbutils.fs.head(log_files_path + "part-00000")` .
- 로그는 사용자 ID, IP 주소, 타임스탬프, 엔드포인트 요청, HTTP 응답 코드 등으로 구성된 공백으로 구분된 텍스트 파일임을 알 수 있습니다.
- **예시 로그 형식:** 10.0.0.1 - user1 [10/Oct/2023:10:00:00 +0000] "GET /api/data HTTP/1.1" 200 1234

```
1 dbutils.fs.head(log_files_path + "part-00000")
```

4.2.3 원시 로그 데이터프레임 생성

- `spark.read.text(log_files_path)` .
- `display(raw_log_files_df)` .display *Spark* (Action) .

```
1 raw_log_files_df = spark.read.text(log_files_path)
2 display(raw_log_files_df)
```

4.3 정규 표현식을 이용한 로그 파싱

로그 데이터는 종종 비정형 텍스트 형태로 제공됩니다. 이를 분석 가능한 구조화된 데이터로 변환하기 위해 정규 표현식(Regular Expression)을 사용합니다.

4.3.1 로그 파싱 함수 정의

- `pyspark.sql.functions.regexp_extract` *IP* , *ID* , , , , .
- 정규 표현식은 복잡할 수 있으며, 시행착오를 통해 패턴을 완성하거나 ChatGPT와 같은 도구의 도움을 받을 수 있습니다. 중요한 것은 생성된 정규 표현식의 각 부분이 무엇을 의미하는지 이해하는 것입니다.

```

1 from pyspark.sql.functions import regexp_extract, col
2
3 # Apache 로그파일파싱을위한정규표현식
4 # 이정규표현식은 IP, 사용자, 타임스탬프, 메서드, 엔드포인트, 프로토콜, 응답코드, 콘
   텐츠키크기를추출합니다.
5 log_pattern = r'^(\S+) (\S+) (\S+) \[[\w:/]+\s[+-]\d{4}\] "(\S+) (\S+)(?:\s
   (\S+))?" (\d{3}) (\S+)$'
6
7 parsed_df = raw_log_files_df.select(
8     regexp_extract(col("value"), log_pattern, 1).alias("ip_address"),
9     regexp_extract(col("value"), log_pattern, 3).alias("user_id"),
10    regexp_extract(col("value"), log_pattern, 4).alias("timestamp_str"),
11    regexp_extract(col("value"), log_pattern, 5).alias("method"),
12    regexp_extract(col("value"), log_pattern, 6).alias("endpoint"),
13    regexp_extract(col("value"), log_pattern, 8).alias("response_code"),
14    regexp_extract(col("value"), log_pattern, 9).alias("content_size")
15 )
16
17 display(parsed_df)

```

4.3.2 파싱된 데이터프레임 확인

- `parsed_df.count()` . 100,000 .
- `parsed_df.createOrReplaceTempView("log_data")` *SQL* . *Spark* .

4.4 사용자 정의 함수(UDF) 생성 및 활용

사용자 정의 함수(UDF)는 Spark에서 제공하는 기본 함수로는 처리하기 어려운 복잡하거나 특정 로직을 직접 정의하여 데이터프레임이나 SQL 쿼리에서 재사용할 수 있도록 하는 기능입니다.

4.4.1 타임스탬프 파싱 문제

- 로그에서 추출한 `timestamp_str` [10/Oct/2023:10:00:00 +0000] .
- `CAST(timestamp_str AS TIMESTAMP)` `NULL` . *Spark* .

```

1 SELECT CAST(timestamp_str AS TIMESTAMP) FROM log_data LIMIT 5;

```

이 쿼리는 `NULL` 값을 반환하거나 오류를 발생시킬 것입니다.

4.4.2 PySpark UDF 생성 및 사용

- **정의:** Python 함수를 정의하고, `pyspark.sql.functions.udf` 데코레이터를 사용하여 Spark UDF로 등록합니다. 이때 반환 타입(Return Type)을 명시해야 합니다.
- **활용:** 데이터프레임 API의 `select` 함수 내에서 UDF를 호출하여 `timestamp_str Unix (EpochTime)` .

```

1 from pyspark.sql.functions import udf

```

```

2 from pyspark.sql.types import FloatType
3 from datetime import datetime
4
5 # 타임스탬프문자열을파싱하는 Python 함수정의
6 def parse_timestamp_udf(timestamp_str):
7     try:
8         # 예시: "10/Oct/2023:10:00:00 +0000" -> datetime 객체
9         dt_object = datetime.strptime(timestamp_str, "%d/%b/%Y:%H:%M:%S %z")
10        return float(dt_object.timestamp()) # Unix 타임스탬프로반환
11    except ValueError:
12        return None
13
14 # Python 함수를 Spark 로UDF 등록반환 ( 타입명시필수 )
15 # FloatType()은 반환값이부동소수점숫자임을나타냅니다 .
16 parse_state_udf = udf(parse_timestamp_udf, FloatType())
17
18 # 데이터프레임에서 API UDF 사용
19 parsed_df.select(parse_state_udf(col("timestamp_str")).alias("unix_timestamp")
20 ).display()

```

4.4.3 SQL UDF 생성 및 사용

- 정의: spark.udf.register를 사용하여 Python 함수를 SQL 컨텍스트에서 사용할 수 있는 UDF로 등록합니다.
- 활용: spark.sql을 통해 SQL 쿼리를 실행할 때, 등록된 UDF를 일반 SQL 함수처럼 호출할 수 있습니다.

```

1 # Python 함수를 SQL 로UDF 등록
2 # 첫번째인자는에서 SQL 사용할 UDF 이름, 두번째는 Python 함수, 세번째는반환타입
3 spark.udf.register("parse_state", parse_timestamp_udf, FloatType())
4
5 # Spark SQL 쿼리에서 UDF 사용
6 clean_logs_df = spark.sql(f"""
7     SELECT
8         ip_address,
9         user_id,
10        FROM_UNIXTIME(parse_state(timestamp_str), 'yyyy-MM-dd HH:mm:ss') AS
11            event_time,
12        method,
13        endpoint,
14        CAST(response_code AS INT) AS response_code,
15        CAST(content_size AS INT) AS content_size
16    FROM log_data
17 """)
18 display(clean_logs_df)

```

위 코드에서는 `FROM_UNIXTIME` Unix `yyyy-MM-dd HH:mm:ss` .

4.5 데이터 정제 및 변환

4.5.1 데이터프레임 컬럼 변환 및 이름 변경

- `content_size`
- `timestamp_str` UDF
- `selectExpr` 또는 `select` 함수를 사용하여 컬럼 이름을 변경 (*Aliasing*)하고 필요한 컬럼만 선택합니다.

```

1 from pyspark.sql.functions import expr
2
3 # 을selectExpr 사용하여 SQL 표현식으로컬럼변환및이름변경
4 clean_logs_df_v2 = parsed_df.selectExpr(
5     "ip_address",
6     "user_id",
7     "FROM_UNIXTIME(parse_state(timestamp_str), 'yyyy-MM-dd HH:mm:ss') AS
        event_time",
8     "method",
9     "endpoint",
10    "CAST(response_code AS INT) AS response_code",
11    "CAST(content_size AS INT) AS content_size"
12 )
13 display(clean_logs_df_v2)

```

4.6 참조 데이터 조인

주요 데이터셋만으로는 모든 정보의 의미를 파악하기 어려울 때가 많습니다. 이때 **참조 데이터(Reference Data)**를 조인하여 데이터를 풍부하게 만들고, 더 깊은 분석을 가능하게 합니다.

4.6.1 HTTP 상태 코드 참조 데이터 로드

- *GitHub*에서 HTTP 상태 코드 목록을 JSON 파일로 가져와 *Databricks* 볼륨에 저장합니다.
- `spark.read.json(path, multiLine=True)`를 사용하여 JSON 파일을 읽어옵니다. `multiLine=True`는 JSON 파일이 여러 줄에 걸쳐 있을 때 필요합니다.

```

1 # 에서GitHub HTTP 상태코드 JSON 파일다운로드및볼륨에저장예시      ()
2 # 실제노트북에서는 dbutils.fs.cp 또는 wget 명령을사용할수있습니다 .
3 # dbutils.fs.cp("dbfs:/Volumes/csi103/lab02/input_data/status_codes.json", "
        dbfs:/Volumes/csi103/lab02/output_data/status_codes.json")
4
5 status_codes_df = spark.read.json(
6     "dbfs:/databricks-datasets/definitive-guide/data/activity-status-codes.
        json",
7     multiLine=True # JSON 파일이여러줄에걸쳐있을경우필요
8 )
9 display(status_codes_df)

```